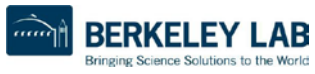
Crash Course in Supercomputing



- Introduction to Parallel Programming Concepts
- Understanding Supercomputer Architecture
- Basic Parallelism & MPI
  - MPI Collectives
- Introduction to OpenMP

Computing π in Parallel

- Ratio of area of square to area of inscribed circle proportional to π

Imagine dartboard with circle of radius R inscribed in square

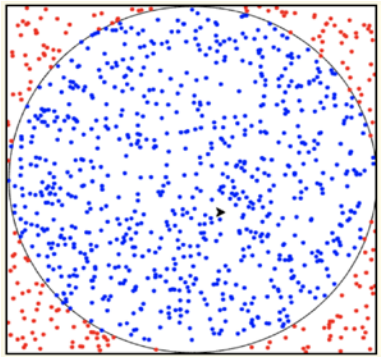
Area of circle $= \pi R^2$

Area of square $= (2R)^2 = 4R^2$

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi R^2}{4R^2} = \frac{\pi}{4}$$

$$\text{Fraction inside circle} \approx \frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi}{4}$$

$$\pi \approx 4 \times \frac{\text{Number of points inside circle}}{\text{Total points}}$$



- **What tasks independent of each other?**

Divide up number of dart throws evenly between processors, so each processor does a share of work

- **What tasks must be performed sequentially?**

Processor 0 will receive tallies from all the other processors, and will compute final value of π

```
dong@pluto2:~/public_html/cs471/lecture/hpc$ cat darts-mpi-v1.c

#include <mpi.h>
#include <stdio.h>

static long num_trials = 1000000;
static long MULTIPLIER = 1366;
static long ADDEND = 150889;
static long PMOD = 714025;

/* Thread-safe LCG random number generator using rank-specific seed */
double lcgrandom_r(unsigned int *seed) {
    long random_next;
    random_next = (MULTIPLIER * (*seed) + ADDEND) % PMOD;
    *seed = random_next;
    return ((double)random_next / (double)PMOD);
}

int main(int argc, char **argv) {
    long i, Ncirc = 0;
    double pi, x, y;

    int rank, size, j, manager = 0;
    MPI_Status status;
    long my_trials, temp;
```

```
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* Determine number of trials for this rank */
    my_trials = num_trials / size;
    if (num_trials % (long)size > (long)rank) my_trials++;

    /* Initialize rank-specific seed */
    unsigned int seed = rank;

    for (i = 0; i < my_trials; i++) {
        x = lcgrandom_r(&seed);
        y = lcgrandom_r(&seed);

        if ((x*x + y*y) <= 1.0)
            Ncirc++;
    }

    /* Reduce results to manager */
    if (rank == manager) {
        for (j = 1; j < size; j++) {
            MPI_Recv(&temp, 1, MPI_LONG, j, j, MPI_COMM_WORLD, &status);
            Ncirc += temp;
        }
        pi = 4.0 * ((double)Ncirc) / ((double)num_trials);
        printf("\n \t Computing pi using six basic MPI functions: \n");
        printf("\t For %ld trials, pi = %f\n\n", num_trials, pi);
    } else {
        MPI_Send(&Ncirc, 1, MPI_LONG, manager, rank, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}dong@pluto2:~/public_html/cs471/lecture/hpc$
```

```c
#include <mpi.h>
#include <stdio.h>

static long num_trials = 1000000;
static long MULTIPLIER = 1366;
static long ADDEND = 150889;
static long PMOD = 714025;

/* Thread-safe LCG random number generator using rank-specific seed */
double lcgrandom_r(unsigned int *seed) {
    long random_next;
    random_next = (MULTIPLIER * (*seed) + ADDEND) % PMOD;
    *seed = random_next;
    return ((double)random_next / (double)PMOD);
}

int main(int argc, char **argv) {
    long i, Ncirc = 0;
    double pi, x, y;

    int rank, size;
    long my_trials;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* Determine number of trials for this rank */
    my_trials = num_trials / size;
    if (num_trials % (long)size > (long)rank) my_trials++;

    /* Initialize rank-specific seed */
    unsigned int seed = rank;

    for (i = 0; i < my_trials; i++) {
        x = lcgrandom_r(&seed);
        y = lcgrandom_r(&seed);

        if ((x*x + y*y) <= 1.0)
            Ncirc++;
    }

    /* Reduce Ncirc from all ranks to the manager (rank 0) */
    long total_Ncirc = 0;
    MPI_Reduce(&Ncirc, &total_Ncirc, 1, MPI_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        pi = 4.0 * ((double)total_Ncirc) / ((double)num_trials);
        printf("\n \t Computing pi using MPI_Reduce: \n");
        printf("\t For %ld trials, pi = %f\n\n", num_trials, pi);
    }

    MPI_Finalize();
    return 0;
}
```

About OpenMP (Open Multi-Processing)

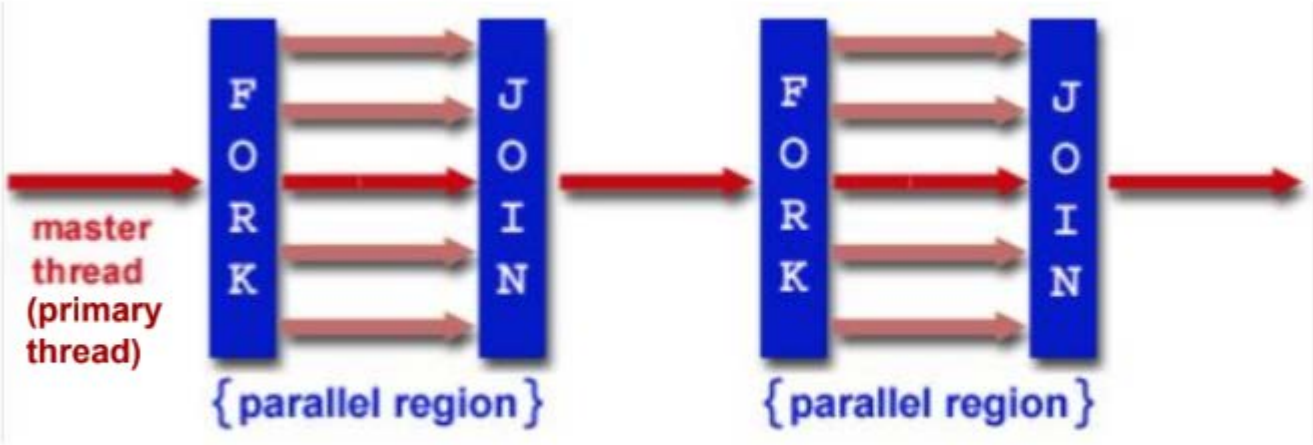
- Industry-standard shared memory programming model
  - Developed in 1997
  - Current standard: 6.0 (November 2024)
- Parallelize small parts of application, one at a time
- Code size grows only modestly
- Single source code for OpenMP and non-OpenMP
  - non-OpenMP compilers simply ignore OMP directives

OpenMP Programming Model

- Application Programmer Interface (API) is combination of
  - Directives (`#pragma omp parallel for`)
  - Runtime library routines (`omp_get_thread_num()`)
  - Environment variables(`OMP_NUM_THREADS=4`)
- API falls into three categories
  - Expression of parallelism (flow control)
  - Data sharing among threads (communication)
  - Synchronization (coordination or interaction)

Parallelism

- Shared memory, thread-based parallelism
- Explicit parallelism (parallel regions)



Source: https://hpc-tutorials.llnl.gov/openmp/

Syntax Overview: C/C++

- Basic format
  - **`#pragma omp`** ***`directive-name`*** *`[clause]`* ***`newline`***
- All directives followed by newline
- Uses pragma construct (pragma = Greek for “thing done”)
- Case sensitive
- Directives follow standard rules for C/C++ compiler directives
- Use curly braces (not on pragma line) to denote scope of directive
- Long directive lines can be continued by escaping newline character with \

#pragma omp parallel

- #pragma tells the compiler it's a compiler-specific instruction.
- omp identifies it as an OpenMP directive.
- The rest (parallel, for, sections, etc.) specifies what kind of parallelism you want.

```
1 #pragma omp parallel
2  { code block
3    a = work(...);
4  }
```

Line 1	Team of threads is formed at parallel region
Lines 2–3	Each thread executes code block and subroutine call, no branching into or out of a parallel region
Line 4	All threads synchronize at end of parallel region (implied barrier)

Ignored by compilers without OpenMP

```
gcc program.c -o program                  # runs serially
gcc -fopenmp program.c -o program       # runs in parallel
```

OPENMP DIRECTIVES: parallel

```
dong@pluto2:~/public_html/cs471/lecture/hpc$ gcc -fopenmp test-openmp.c -o test-openmmp
dong@pluto2:~/public_html/cs471/lecture/hpc$ ./test-openmp
Hello from thread 10 of 24
Hello from thread 20 of 24
Hello from thread 6 of 24
Hello from thread 8 of 24
Hello from thread 14 of 24
Hello from thread 2 of 24
Hello from thread 0 of 24
Hello from thread 12 of 24
Hello from thread 16 of 24
Hello from thread 22 of 24
Hello from thread 4 of 24
Hello from thread 18 of 24
Hello from thread 23 of 24
Hello from thread 9 of 24
Hello from thread 1 of 24
Hello from thread 13 of 24
Hello from thread 3 of 24
Hello from thread 15 of 24
Hello from thread 17 of 24
Hello from thread 5 of 24
Hello from thread 21 of 24
Hello from thread 7 of 24
Hello from thread 11 of 24
Hello from thread 19 of 24
dong@pluto2:~/public_html/cs471/lecture/hpc$ export OMP_NUM_THREADS=4
dong@pluto2:~/public_html/cs471/lecture/hpc$ ./test-openmp
Hello from thread 0 of 4
Hello from thread 3 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
dong@pluto2:~/public_html/cs471/lecture/hpc$
```

- A block of code executed by multiple threads

```
dong@pluto2:~/public_html/cs471/lecture/hpc$ cat test-openmp.c
// test_openmp.c
#include <omp.h>
#include <stdio.h>

int main() {
    #pragma omp parallel
    printf("Hello from thread %d of %d\n", omp_get_thread_num(), omp_get_num_threads());
    return 0;
}dong@pluto2:~/public_html/cs471/lecture/hpc$
```

Order of execution is scheduled by OS!!!

Compiling and Running openMP Programs

gcc –fopenmp test-openmp.c -o test-openmp

- Use the gcc –fopemp to compile the program file test-openmp.c
- Create an executable named test-openmp

export OMP_NUM_THREADS=4

./test-openmp

- Set the number of threads to be 4 → run it using 4 threads (No space around =)
- ./test-openmp → this is the program to run.

OpenMP Directives: Scheduling

Default scheduling determined by implementation

- Static
  - ID of thread performing particular iteration is function of iteration number and number of threads
  - Statically assigned at beginning of loop
  - Best for known, predictable amount of work per iteration
  - Low overhead
- Dynamic
  - Assignment of threads determined at runtime (round robin)
  - Each thread gets more work after completing current work
  - Load balance is possible for variable work per iteration
  - Introduces extra overhead

Which Loops are Parallelizable?

- Number of iterations known upon entry, and does not change
- Each iteration independent of all others
- No data dependence
- Trick: If a loop can be run backwards and get the same results, then it is almost always parallelizable!

General form:

```
1 #pragma omp parallel for
2  for (i=0; i<N; i++)
3  {
4    a[i] = b[i] + c[i];
5  }
6
```

```
#pragma omp parallel
{
  #pragma omp for
  for (i=0; i<N; i++)
  {a[i] = b[i] + c[i];}
}
```

Line 1 Team of threads formed (parallel region).

Lines 2–6 Loop iterations are split among threads.
Implied barrier at end of block(s) {}.

```
#include <stdio.h>
#include <omp.h>
#define N 1000

int main(void) {
    int i;
    double a[N], b[N];

    /* Parallel initialization using OpenMP *,
    #pragma omp parallel for
    for (i = 0; i < N; i++) {
        a[i] = i * 1.5;
        b[i] = i + 22.35;
    }

    /* Print from a single thread */
    #pragma omp single
    {
        printf("a[0] = %.2f, b[0] = %.2f\n", a[0], b[0]);
        printf("a[N-1] = %.2f, b[N-1] = %.2f\n", a[N-1], b[N-1]);
        printf("Number of threads used: %d\n", omp_get_max_threads());
    }
    return 0;
}
```

OpenMP Directives: Sections

- Non-iterative work-sharing construct
- Divide enclosed sections of code among threads

```c
#include <omp.h>
#define N 1000

int main() {
    int i;
    double a[N], b[N];
    double c[N], d[N];

    /* Parallel initialization using OpenMP */
    #pragma omp parallel for private(i)
    for (i = 0; i < N; i++) {
        a[i] = i * 1.5;
        b[i] = i + 22.35;
    }
```

```c
    #pragma omp parallel shared(a,b,c,d) private(i)
    {
        #pragma omp sections nowait
        {
            #pragma omp section
            for (i = 0; i < N; i++)
                c[i] = a[i] + b[i];

            #pragma omp section
            for (i = 0; i < N; i++)
                d[i] = a[i] * b[i];
        } /* end of sections */
    } /* end of parallel section */

    return 0;
}
```

OpenMP Directives: Synchronization

- Sometimes, need to make sure threads execute regions of code in proper order
  - Maybe one part depends on another part being completed
  - Maybe only one thread need execute a section of code
- Synchronization directives
  - **Critical**: Specifies section of code that must be executed by only one thread at a time
  - **Barrier**: Synchronizes all threads: thread reaches barrier and waits until all other threads have reached barrier, then resumes executing code following barrier
  - **Single**: Enclosed code is to be executed by only one thread

About Variable Scope

- Variables can be shared or private within a parallel region
- Shared: one copy, shared between all threads
  - Single common memory location, accessible by all threads
- Private: each thread makes its own copy
  - Private variables exist only in parallel region

- Private starting with defined values: firstprivate(list)
  - Private variables initialized to be the value held immediately
  - before entry into parallel region

- Private ending with defined value: lastprivate(list)
  - At end of loop, set variable to value set by final iteration of loop

About Variable Scope

- By default, all variables shared except
  - Index values of parallel region loop – private by default
  - Local variables and value parameters within subroutines called within parallel region – private
  - Variables declared within lexical extent of parallel region –private

- Variable scope is the most common source of errors in OpenMP codes
  - Correctly determining variable scope is key to correctness and performance of your code

OpenMP Environment Variables

- Set environment variables to control execution of parallel code
- OMP_SCHEDULE
  - Determines how iterations of loops are scheduled
  - E.g., export OMP_SCHEDULE=”dynamic, 4”
- OMP_NUM_THREADS
  - Sets maximum number of threads
  - E.g., export OMP_NUM_THREADS=4

OpenMP Reduction

- Recall previous example of parallel dot product
- – Simple parallel-for doesn't work due to race condition on shared sum
- – Best solution is to apply OpenMP's reduction clause
- – Doing private partial sums is fine too; add a critical section for sum of ps

```
// repetitive updates: oops
#pragma omp parallel for
for (i=0; i<N; i++)
     sum = sum + b[i]*c[i];
```

```
// repetitive reduction: OK
#pragma omp parallel for \
     reduction(+:sum)
for (i=0; i<N; i++)
     sum = sum + b[i]*c[i];
```

```
// repetitive updates: OK
int ps = 0;
#pragma omp parallel \
      firstprivate(ps)
{
#pragma omp for
  for (i=0; i<N; i++)
     ps = ps + b[i]*c[i];
#pragma omp critical
  sum = sum + ps; }
```

Computing π in Parallel

- Ratio of area of square to area of inscribed circle proportional to π

Imagine dartboard with circle of radius R inscribed in square

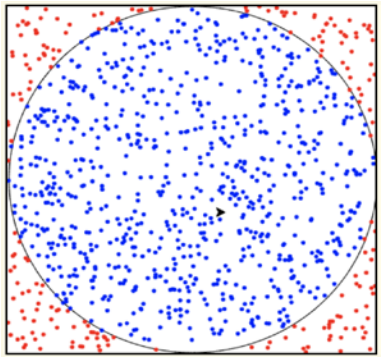
Area of circle $= \pi R^2$

Area of square $= (2R)^2 = 4R^2$

$$\frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi R^2}{4R^2} = \frac{\pi}{4}$$

$$\text{Fraction inside circle} \approx \frac{\text{Area of circle}}{\text{Area of square}} = \frac{\pi}{4}$$

$$\pi \approx 4 \times \frac{\text{Number of points inside circle}}{\text{Total points}}$$



- **What tasks independent of each other?**

Divide up number of dart throws evenly between processors, so each processor does a share of work

- **What tasks must be performed sequentially?**

Processor 0 will receive tallies from all the other processors, and will compute final value of π

```c
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

static long num_trials = 10000000;
static long MULTIPLIER = 1366;
static long ADDEND = 150889;
static long PMOD = 714025;

/* Thread-safe linear congruential random number generator */
double lcgrandom_r(unsigned int *seed) {
    long random_next;
    random_next = (MULTIPLIER * (*seed) + ADDEND) % PMOD;
    *seed = random_next;
    return ((double)random_next / (double)PMOD);
}
```

```c
int main(int argc, char **argv) {
    long i;
    long Ncirc = 0;
    double pi, x, y;
    double r = 1.0;   /* radius of circle */
    double r2 = r * r;

    #pragma omp parallel
    {
        /* Unique seed per thread */
        unsigned int seed = omp_get_thread_num();

        #pragma omp for private(x, y) reduction(+:Ncirc)
        for (i = 0; i < num_trials; i++) {
            x = lcgrandom_r(&seed);
            y = lcgrandom_r(&seed);

            if ((x * x + y * y) <= r2)
                Ncirc++;
        }
    }

    pi = 4.0 * ((double)Ncirc) / ((double)num_trials);

    printf("\n\tComputing pi using OpenMP:\n");
    printf("\tFor %ld trials, pi = %f\n\n", num_trials, pi);

    return 0;
}
```

Reference Links

- https://www.nersc.gov/news-and-events/calendar-of-events/hpc-crash-course-jun2025
- https://www.cac.cornell.edu/education/training/StampedeJan2017/IntroOpenMPandMPIwithKNL.pdf

Hybrid MPI + openMP

```c
/* Compute pi using hybrid MPI/OpenMP */
#include "lcgenerator.h"
#include <mpi.h>
#include <omp.h>
#include <stdio.h>

static long num_trials = 1000000;

int main(int argc, char **argv) {
  long i;
  long Ncirc = 0;
  double pi, x, y;
  double r = 1.0; /* radius of circle */
  double r2 = r*r;

  int rank, size, manager = 0;
  MPI_Status status;
  long my_trials, temp;
  int provided;

  MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
  MPI_Comm_size(MPI_COMM_WORLD, &size);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  my_trials = num_trials/size;
  if (num_trials%(long)size > (long)rank) my_trials++;
  random_last = rank;

  #pragma omp parallel
  {
  #pragma omp for private(x,y) reduction(+:Ncirc)
    for (i = 0; i < my_trials; i++) {
  #pragma omp critical (randoms)
  {
      x = lcgrandom();
      y = lcgrandom();
  }

      if ((x*x + y*y) <= r2)
        Ncirc++;
    }
  }

  MPI_Reduce(&Ncirc, &temp, 1, MPI_LONG, MPI_SUM, manager, MPI_COMM_WORLD);

  if (rank == manager) {
    Ncirc = temp;
    pi = 4.0 * ((double)Ncirc)/((double)num_trials);
    printf("\n \t Computing pi using hybrid MPI/OpenMP: \n");
    printf("\t For %ld trials, pi = %f\n", num_trials, pi);
    printf("\n");
  }
  MPI_Finalize();
  return 0;
}
```

To compile and run

- To compile

mpicc -fopenmp -o darts-hybrid darts-hybrid.c –lm

- To run
  - export OMP_NUM_THREADS=4
  - mpirun –np 2 darts-hybrid